MAD3OOM



THE MAD3OOM WHITE PAPER

Intelligent Customer Support & Unified Customer Operations

الورقة البيضاء الأساسية لمتصة مدعوم

PRODUCT VISION • PLATFORM ARCHITECTURE • ROADMAP

October 2026 | Version 1.0

About this document

ITEM	DETAIL
Title	**The Mad3oom White Paper** — Intelligent Customer Support & Unified Customer Operations
Edition	Public edition, version 1.0, October 2026. English and Arabic editions, equivalent in meaning and structure.
Information as of	9 October 2026, from the project's engineering documentation and source code.
Intended readers	Prospective partners, technical stakeholders and business audiences
Product stage	Pre-launch. The official launch is still ahead.

Purpose and scope

This paper presents the product vision, platform architecture and roadmap of Mad3oom, an intelligent customer support and customer operations platform. It is written as a credible foundation for conversations with partners and technical stakeholders, and it separates what exists from what is under development, planned or merely envisaged.

How to read the status labels

Every capability in this paper carries one of the labels below. They describe engineering status, not commercial availability.

EXISTING	Implemented in the project repositories, with automated tests where noted. It does not imply production maturity, wide use or independent validation.
IN DEVELOPMENT	Active engineering work: code written or partly delivered, not yet complete or enabled for users.
PLANNED	Designed and intended, with no delivered implementation yet. Subject to validation and priorities.
FUTURE	A long-term opportunity or exploratory direction. No commitment is made.
RECOMMENDATION	Architectural direction proposed by this paper. It is advice, not a statement about the current system.

NOTICE

Statements about the future are intentions that depend on technical validation and business priorities. They are not commitments, and no delivery dates are given. This paper reports no customer counts, revenue, market share, security certifications, partnerships or measured real-world accuracy, because none are documented. See Appendix C for the evidence basis. This public edition describes product capabilities, architecture at a conceptual level and security principles, and does not publish operational or implementation detail.

Contents

Chapter 1 **Executive Summary** 6
1.1 What Mad3oom is 6
1.2 Whom it serves 6
1.3 The problems it addresses 7
1.4 Main components 7
1.5 The value of one connected workflow 8
1.6 Where the project stands 8

Chapter 2 **Vision and Problem Statement** 9
2.1 The vision 9
2.2 Six problems in customer support 9
2.3 What the problems imply for design 10

Chapter 3 **The Mad3oom Solution** 11
3.1 A unified approach 11
3.2 How the parts are meant to relate 12
3.3 Two kinds of work, kept separate 13

Chapter 4 **Mad3oom Workspace** 14
4.1 The idea 14
4.2 What exists and what does not 14
4.3 How it is built 15
4.4 Integrity and access rules 16
4.5 Engineering evidence and limits 16

Chapter 5 **Mad3oom Relay** 17
5.1 The business problem 17
5.2 What a continuity record is 17
5.3 How Relay complements ticketing 18
5.4 Capabilities against the intended concepts 19
5.5 Governance of evidence 20
5.6 Stage and limits 21

Chapter 6 **SIE: Support Intelligence Engine** 22
6.1 What SIE is, and what it is not 22
6.2 Interpreting a problem versus coordinating the work 22
6.3 A layered design 23
6.4 Why the layers matter 24
6.5 What has been verified, and what has not 25
6.6 Where SIE runs 25

Chapter 7 **Integrations and Connectivity** 26
7.1 Integration principles 26
7.2 Channels and connected systems 27
7.3 WhatsApp Cloud API: three different things 28
7.4 What is not claimed 29

Chapter 8 **Technical Architecture** 30
8.1 Conceptual architecture 30
8.2 Principles at work 31
8.3 Quality practice 31

Chapter 9 **Security, Privacy and Governance** 32
9.1 Security by design 32
9.2 Safeguards and how they are substantiated 33
9.3 Governance and continuous improvement 33

Chapter 10 **Product Differentiation and Value** 34
10.1 Five design principles 34
10.2 How the parts combine: an illustrative journey 35
10.3 Measures that would show the value 35
10.4 What is not claimed 35

Chapter 11 **Product Roadmap** 36
11.1 Existing foundations 37
11.2 Current development and refinement 37
11.3 Planned capabilities 38
11.4 Long-term opportunities 38

Chapter 12 **Conclusion** 39

Appendix A **Capability Status Register** 40

Appendix B **Glossary** 42

Appendix C **Basis of Preparation and Limitations** 43
C.1 Sources reviewed 43
C.2 Method 43
C.3 What was not verified 43
C.4 Editorial rules 44
C.5 Document history 44

01 CHAPTER 1

Executive Summary

Mad3oom is a platform for technical support and customer operations. It brings tickets, customer conversations, customer records, knowledge and follow-up work into one structured environment, and adds a layer of support-specific intelligence on top.

AT A GLANCE

- Mad3oom unifies tickets, conversations, customer records, knowledge and operational follow-up in one platform.
- Three layers shape its design: **Workspace** for multi-context work, **Relay** for continuity and accountability, and **SIE** for support intelligence.
- Foundations exist in code; the official launch is still ahead. Every capability in this paper is labeled by stage.
- The architecture favors modularity, clear interfaces and explainable decisions over feature breadth.

1.1 What Mad3oom is

Mad3oom is a software platform for support teams. It lets a team receive customer requests, understand them, assign responsibility, keep the context of every interaction and follow each request until it is resolved. It is designed as a coherent technology product with a defined architecture, rather than as a chat widget or a conventional ticket list.

The design is Arabic-first: interfaces are right-to-left, and the intelligence layer is built to handle colloquial Arabic, with a focus on Egyptian usage, and Arabic written in Latin letters, while the engine can also reply in English.

1.2 Whom it serves

- **Support agents and supervisors**, who need one place to work on tickets and conversations without losing context.
- **Companies** that support their own customers and need structure, clear ownership and visibility over open work.
- **Customers**, who reach support through a portal and messaging channels and should not have to repeat themselves.

- **Platform operators**, who need governance over access, data and automation.

1.3 The problems it addresses

Support work degrades when customer information is scattered, context is lost between tools, follow-ups are forgotten, handoffs are incomplete, ownership is unclear and intelligence is applied inconsistently. These are common industry problems, analyzed in Chapter 2. They are described here as the reasoning behind the design, not as findings of a completed customer study.

1.4 Main components

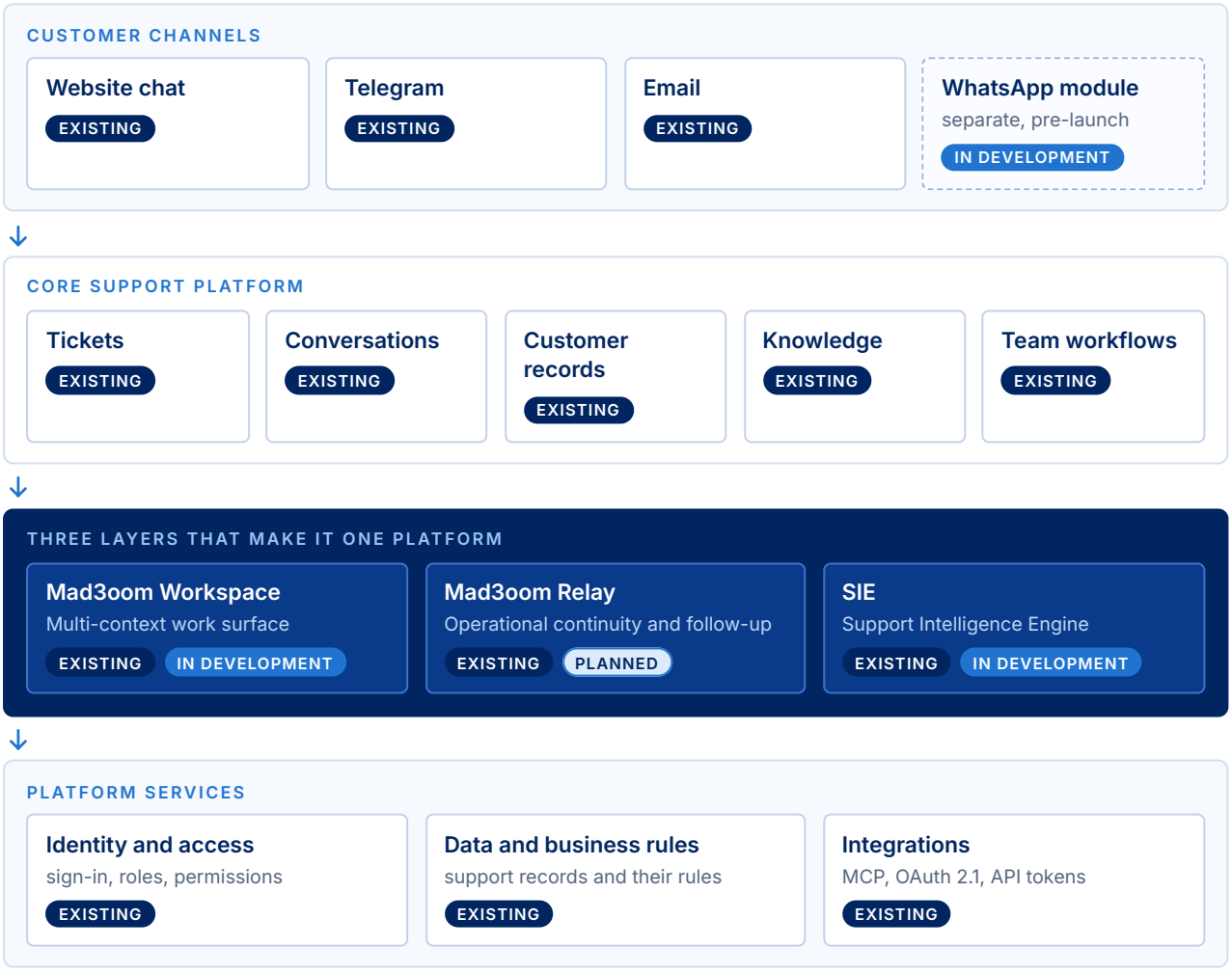


Figure 1. Mad3oom at a glance. A conceptual map of the platform, with the stage of each part. It is not a deployment diagram.

Table 1. Components and their stage. Workspace, Relay and SIE each combine delivered parts with parts still in development or planned; the detail is in Chapter 4, Chapter 5 and Chapter 6.

COMPONENT	ROLE	STAGE
Core support platform	Tickets with SLA targets and notifications, a helpdesk inbox with human–bot handoff, customer history, a knowledge base and team workflows.	EXISTING
Mad3oom Workspace	A tabbed, split-panel surface that hosts tickets, conversations and customer records side by side.	EXISTING IN DEVELOPMENT
Mad3oom Relay	Continuity records that keep evidence, ownership, a next action and a deadline together.	EXISTING IN DEVELOPMENT PLANNED
SIE	A deterministic engine that interprets customer messages and decides the next support action.	EXISTING IN DEVELOPMENT
Integration layer	Channel adapters, MCP with OAuth 2.1, API tokens and event-driven side effects.	EXISTING

1.5 The value of one connected workflow

The intended value comes from three properties that separate tools rarely provide together.

- **Context travels with the work.** A ticket, the conversation behind it and the customer's history are reachable from the same place.
- **Responsibility is explicit.** Follow-up work carries a next action and a deadline, and either one accountable owner or an explicit “unassigned” marker, all visible to the team.
- **Intelligence is controlled and explainable.** The engine chooses from a closed set of actions through ordered rules, and records why.

These are design intentions. Whether they translate into fewer missed requests or faster resolution can only be shown by measurement in real use, and this paper reports no such measurements.

1.6 Where the project stands

> STAGE OF THE PROJECT
>
> Mad3oom has not launched. The paper describes foundations that exist in the code, work that is under way and directions that are planned. A controlled pilot is the intended first step. No customer deployments, benchmarks of real-world accuracy or security certifications are claimed.

02

CHAPTER 2

Vision and Problem Statement

The vision is simple to state and demanding to deliver: technical support that is organized, context-aware, accountable and intelligent. The obstacles are not unique to Mad3oom. They are the recurring failure points of support operations assembled from disconnected tools.

2.1 The vision

Mad3oom aims to make technical support more organized, more context-aware, more accountable and more intelligent. The order is deliberate and reflects design logic, not a delivery schedule. Organization comes first, because intelligence applied to disorganized work produces confident mistakes. Context follows, because every later improvement depends on knowing what has already happened. Accountability comes next, because work without an owner is the main source of dropped requests. Intelligence is the last layer.

2.2 Six problems in customer support

> NATURE OF THIS ANALYSIS
>
> These are recurring industry problems that the platform is designed to address. They are drawn from general support-operations practice and from the design reasoning in Mad3oom's engineering documents. They are not quantified findings from a completed customer study, and this paper reports no statistics about them.

1. Customer information is scattered

One customer can exist as a chat session, a ticket, a profile and an email thread, each in a different screen or product. Agents rebuild the picture by hand, and different agents rebuild it differently.

2. Context is lost when switching

Moving from a conversation to a ticket to the customer's history usually means leaving one view to open another, and half-written replies, the reasoning behind a decision and the reader's place in a long thread are left behind.

3. Follow-ups are forgotten

A commitment made in a conversation, such as “I will check and come back tomorrow”, often lives only in the message itself. Unless someone copies it into a task list, it depends on memory, and unresolved requests quietly age.

4. Handoffs are incomplete

When responsibility moves between people or shifts, the receiving agent often has the case but not the context: what was promised, what was already tried and what is still unknown.

5. Ownership, status and next action are hard to see

Supervisors can see that work is open, but not always who owns it, what the next step is or whether a deadline has passed. Work with no owner is invisible until a customer complains.

6. Intelligence is hard to apply consistently

Automation and AI features added tool by tool behave differently across channels, are difficult to explain afterwards and hard to test. Without a controlled design, intelligence amplifies inconsistency instead of reducing it.

2.3 What the problems imply for design

Each problem points to a design requirement. The table shows the response chosen for each one, the component that carries it and the stage that response has reached.

Table 2. Problems, design responses and stage. Where two stages appear, the first covers what exists and the second what is in development or planned.

PROBLEM	DESIGN RESPONSE	CARRIED BY	STAGE
Scattered information	Tickets, conversations and customer records in one platform, shown side by side.	Core platform, Workspace	EXISTING
Lost context	Several work contexts open at once; unsent text is protected when tabs move or close.	Workspace	EXISTING
Forgotten follow-ups	Records with a next action, a deadline and a derived overdue flag. Scheduled reminders and escalation are planned.	Relay	EXISTING PLANNED
Incomplete handoffs	Controlled reassignment with a history of changes. Handovers that the receiver must accept, clarify or decline are planned.	Relay	EXISTING PLANNED
Limited visibility	Lists filtered by owner, status and category, with unassigned work flagged. A continuity monitor that explains why a record needs attention is planned.	Relay	EXISTING PLANNED
Inconsistent intelligence	One layered, deterministic engine shared across channels, with a recorded reason for each decision.	SIE	EXISTING IN DEVELOPMENT

These responses explain the structure of the platform described in Chapter 3.

03

CHAPTER 3

The Mad3oom Solution

Mad3oom's answer is not another silo. It is a connected set of parts: the records of support work, a surface to work on them, a layer designed to keep work from being dropped, and an engine that understands customer messages.

3.1 A unified approach

The platform treats six things as parts of one workflow rather than as separate tools. Each is described below as it exists today in the project's repositories.

Table 3. The six elements of the unified approach and the stage each has reached.

ELEMENT	WHAT IT PROVIDES	STAGE
Support tickets	A ticket lifecycle from opening to resolution, in which customers can close a ticket or explicitly reopen a resolved one; SLA targets; assignment and distribution; in-platform notifications, ticket email templates, and Telegram alerts for staff.	EXISTING
Customer conversations	Website chat and Telegram feed a helpdesk inbox with teams, assignment, tags, internal notes, attachments, scheduled replies and an explicit hand-off between the bot and a human agent.	EXISTING
Customer records	A customer history page that gathers a customer's tickets and conversations in one view.	EXISTING
Knowledge resources	A public knowledge base and help center, plus knowledge entries that the intelligence engine can attach to answers.	EXISTING
Team workflows	Roles for platform staff and companies, an automation builder for rules and actions, and company accounts with their own dashboards.	EXISTING
Operational follow-up	Mad3oom Relay: continuity records with source evidence, one owner or an explicit “unassigned”, and, for follow-ups, a next action and a deadline. See Chapter 5.	EXISTING PLANNED

3.2 How the parts are meant to relate

Three layers sit on top of the core records. **Workspace** is the surface on which agents see those records together. **Relay** is the layer that keeps follow-up work attached to its evidence, its owner and its deadline. **SIE** is the engine that interprets what customers write. The figure below shows the intended relationships, and the table that follows states which of them are implemented today.

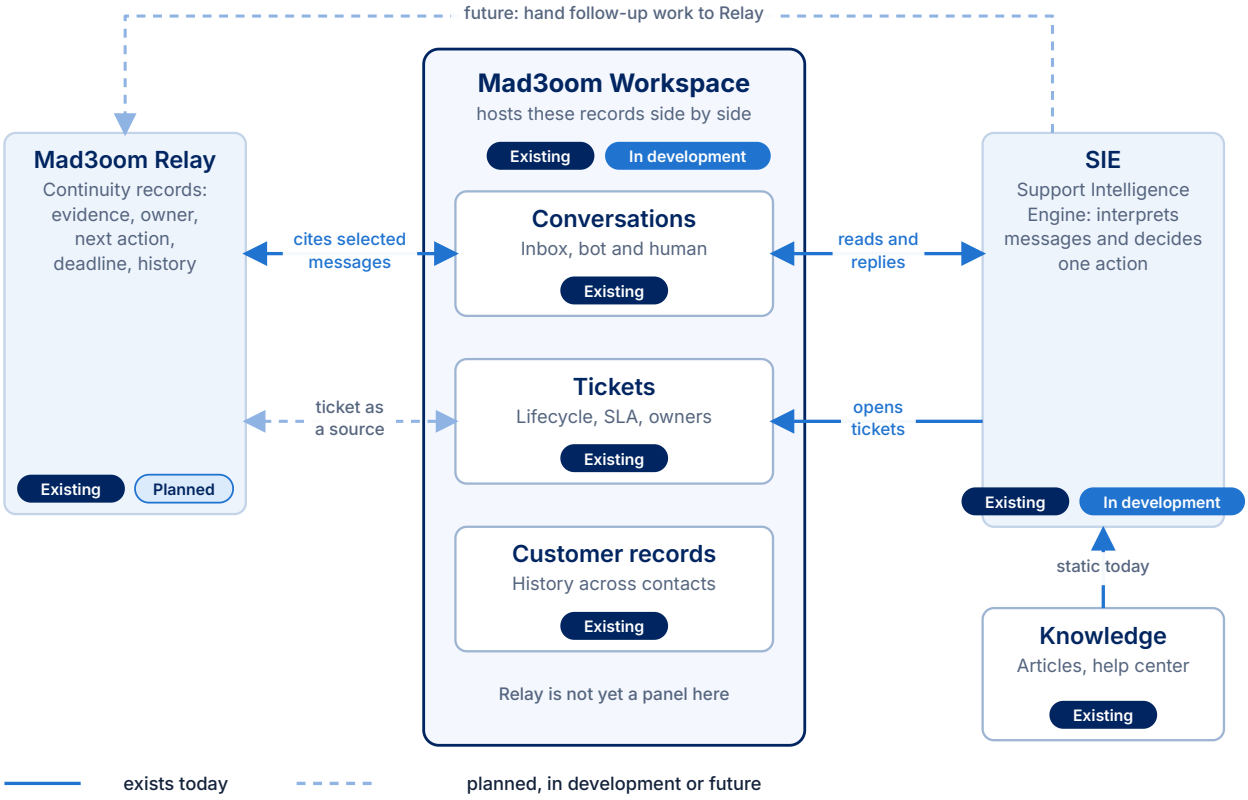


Figure 2. Intended relationships between the parts. Solid lines exist today; dashed lines are planned, in development or future. This is a conceptual map, not a data-flow specification.

Table 4. Relationships between the parts and their stage.

RELATIONSHIP	WHAT IT MEANS	STAGE
Workspace and the hosted pages	The existing pages are hosted as panels; they keep their own logic and access rules.	EXISTING
Relay and conversations	An agent selects messages in a conversation and creates a continuity record that cites them.	EXISTING
Relay and tickets	A ticket as a source of a continuity record.	PLANNED
Relay inside Workspace	A Relay record opened as a panel beside the conversation it came from.	PLANNED
SIE and conversations	The engine reads inbound website and Telegram messages and writes its replies; a human takeover stops it.	EXISTING
SIE and tickets	When the decision is to open a ticket, the message, state and ticket are committed together.	EXISTING
Knowledge and SIE	Static knowledge entries are attached to answers. Grounding in live account data and published knowledge is in development.	EXISTING IN DEVELOPMENT
SIE and Relay	The engine handing a follow-up to Relay. No such link exists today.	FUTURE

LIMITATIONS

Not every integration or workflow described in this paper is fully implemented. Where a relationship is marked as planned, in development or future, the platform does not yet behave that way. The rest of the paper keeps the same distinction.

3.3 Two kinds of work, kept separate

A central design decision is to keep **interpretation** apart from **coordination**. Interpretation answers the question “what is this customer telling us, and what should happen next in this conversation?”. That is SIE's job. Coordination answers “who is responsible for what, by when, and with which evidence?”. That is Relay's job, together with ticketing. Keeping them apart lets each be built, tested and improved on its own, and is designed to keep an intelligent component from silently taking decisions that belong to people.

04

CHAPTER 4

Mad3oom Workspace

Support work rarely fits on one screen. Mad3oom Workspace lets an agent keep several tickets, conversations and customer records open at once, arranged as tabs and split panels, without losing what has been typed or where they left off.

4.1 The idea

Workspace is a multi-tab work surface inside the Mad3oom administration console. Rather than treating the inbox, the tickets list and the customer history as separate pages that replace one another, it hosts them side by side. An agent can read a conversation, open the related ticket beside it and look at the customer's history in a third pane. Each stays as it was left.

4.2 What exists and what does not

Table 5. Workspace capabilities by stage. A row with no capability name continues the row above.

CAPABILITY	DETAIL	STAGE
Tickets, conversations and customers in one place	Six panel types: the inbox, tickets and customer-history pages, and single-conversation, single-ticket and single-customer panels.	EXISTING
Switching without losing context	Hosted pages keep running while hidden, within the live-panel limit described below. Closing a tab that holds unsent text asks for confirmation; moving or docking a tab never reloads it.	EXISTING
Organizing views	Tabs, groups and nested splits made by dragging a tab to any edge or the center; resizable dividers; quick open with Ctrl+K or Cmd+K; starter layouts; reopening the last closed tab.	EXISTING
Opening related records beside the source	Following a link from a ticket to its customer opens the customer in another group, or splits the view if only one group exists.	EXISTING
Layouts that survive a reload	Saved per device in browser storage, with titles resolved live and never stored.	EXISTING
	Saving each agent's layout on the server, so it follows the agent across devices. This is in development; until then, layouts are saved per browser.	IN DEVELOPMENT
Keyboard, RTL and small screens	Tab-list semantics, direction-aware arrow keys, menu alternatives to dragging, mirrored layout in Arabic, and a one-panel compact mode below 900 px.	EXISTING

CAPABILITY	DETAIL	STAGE
Relay as a panel	Opening Relay records as a first-class panel type next to the conversation they cite.	PLANNED
More panel types	Knowledge and reporting panels. The registry is designed to accept new types without changes to the layout engine.	PLANNED
Adaptive organization	Named layouts for recurring tasks and context shared across panels.	FUTURE

4.3 How it is built

The inbox and tickets pages are built to run once per page, and rewriting the tested helpdesk code was ruled out. Workspace therefore **hosts the existing pages** as panels, each keeping its own logic, access rules and live updates. A tab behaves like a browser tab of that page.

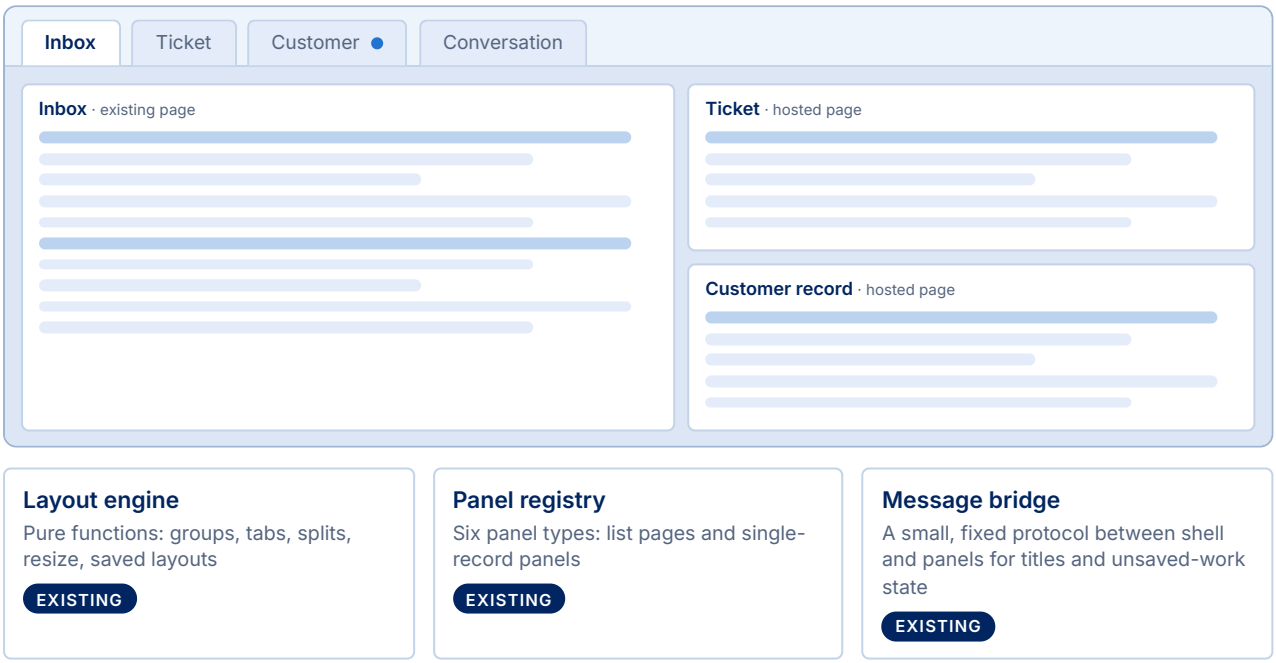


Figure 3. The Workspace model. A shell arranges tabs and split groups; each panel is an existing page. The three supporting parts are shown below the window. The window is illustrative, not a screenshot.

- **Positioned panels.** Hosted pages are placed over the area of the group that shows them rather than moved in the page structure, because moving a hosted page would reload it and discard an unsent reply.
- **A pure layout model.** Tabs, groups and splits are described by plain data and pure functions, tested without a browser. Edges are logical (start and end), so right-to-left is correct by construction. Limits on the number of panels, groups and nesting depth keep layouts bounded.
- **A panel registry.** A registry of allowed panel types validates their parameters.
- **A message bridge.** The shell and hosted pages exchange titles and unsaved-work state through a small, fixed protocol.

4.4 Integrity and access rules

Workspace is designed to follow these rules.

- Moving, docking or splitting a tab never reloads or recreates its hosted page.
- Closing a tab, closing others, resetting the layout or leaving the page checks for unsent work first. The cap on live panels never unloads a panel that holds unsent work.
- Workspace does not submit support actions itself. Replies, ticket changes and other support writes happen inside the hosted page, so the shell does not duplicate a submission.
- Saved layouts are validated, and each record panel is checked against the user's own access before it opens; a record that is no longer visible appears as an unavailable tab.
- A saved layout contains only panel types and record identifiers, never names, email addresses or message text.
- If saving fails, Workspace keeps working with the in-memory layout, and every page remains usable on its own.

4.5 Engineering evidence and limits

Table 6. Development-environment checks. They show that the design works as specified; they say nothing about how Workspace performs for agents on production data.

CHECK (9 OCTOBER 2026)	RESULT
Layout engine unit tests	28 of 28 pass
Browser render tests of the real page code on test doubles	22 of 22 pass
Layout operations at the panel limit	under 1 ms each
Re-render on tab switch, typical layout (6 tabs)	median about 9 ms

LIMITATIONS

Each live panel is a full page instance with its own real-time connection, so the number of live panels is capped at six and the least recently shown clean panel is unloaded. A draft typed in the inbox does not follow a conversation opened as its own tab. Hosted pages keep their existing language while the shell follows the language switch. Drag-and-drop is not supported on touch screens, where compact mode is used instead. No automated accessibility audit has been run. Saving layouts on the server is in development, so layouts are saved per browser until then.

05

CHAPTER 5

Mad3oom Relay

Tickets record cases. Relay records commitments. It is the follow-up and coordination layer that keeps a piece of work attached to its evidence, its owner and its deadline, so that a promise made in a conversation does not depend on someone remembering it.

5.1 The business problem

Work loses its context, its owner or its next action when responsibility moves between people or shifts. The usual symptoms are forgotten follow-ups, problems that outlive a shift, handovers without context, commitments discussed in chat and never tracked, and customers who must explain their issue again. A ticketing system answers whether a case is open. It does not by itself answer who has promised what, by when, on the strength of which evidence.

5.2 What a continuity record is

> DEFINITION
>
> A **continuity record** is the operational unit of Relay. It combines evidence (references to selected messages), one accountable owner or an explicit “unassigned”, a next action, a deadline with an explicit time zone, a completion that must be justified, and an append-only history.

Today a record is either a **follow-up**, which must have a next action and a deadline, or an **issue**, a problem that needs resolving. A third kind, **handover**, is planned (see below). Each record can also carry one of nine categories, such as order status, payment and billing, technical issue or complaint.

5.3 How Relay complements ticketing

Table 7. Ticketing and Relay answer different questions and are designed to be used together.

	TICKET	RELAY RECORD
Main question	Is this customer case open, and how is it progressing?	Who committed to what, by when, on which evidence?
Scope	A customer-facing case.	Any piece of follow-up work, whether or not a ticket exists.
Evidence	The messages of the ticket thread.	Selected messages cited from conversations, shown only to people who still have access to the original conversation.
Time	SLA targets by priority.	An explicit deadline and time zone. Overdue is derived and never resolves anything by itself.
Ownership	An assigned agent or team.	One accountable owner or an explicit “unassigned”, with rules on who may assign.
History	Ticket activity.	An append-only event log that holds identifiers and changes, not message text.

5.4 Capabilities against the intended concepts

Table 8. Relay capabilities against the six intended concepts. A row with no concept name continues the row above.

CONCEPT	WHAT EXISTS OR IS INTENDED	STAGE
Tracking follow-ups and outstanding actions	Follow-up and issue records with a lifecycle (open, scheduled, in progress, waiting, resolved, cancelled), a next action, a priority and a category; a filterable list; creation from selected inbox messages through a three-step dialog.	EXISTING
Preserving context through summaries	Each record can hold a summary written by the person who creates it, and cites its source messages. Excerpts are shown only to people who still have access to the original conversation. Automatic summarization is not implemented.	EXISTING
	Assisted extraction of titles, dates and next actions, with every suggestion editable and never confirmed automatically.	FUTURE
Clarifying ownership	One owner field; unassigned work is flagged. Assigning work to others is a controlled permission granted explicitly; it is not a default.	EXISTING
Supporting handoffs	Reassignment under the rules above, with a history of every change.	EXISTING
	A formal handover that the receiver must accept, query or decline; handovers to a team where the first to accept wins; end-of-shift bundles. The record keeps the previous owner until acceptance.	PLANNED
Making pending and overdue work visible	An overdue flag derived from the deadline, shown in lists; filters by status, owner and category.	EXISTING
	Reminders before a deadline, escalation, and a continuity monitor that lists records needing attention with a reason (unassigned, no next action, overdue, handover pending, owner inactive).	PLANNED
Continuity and accountability	An append-only event log; version checks that reject stale edits; idempotency keys that prevent duplicate records on retries.	EXISTING

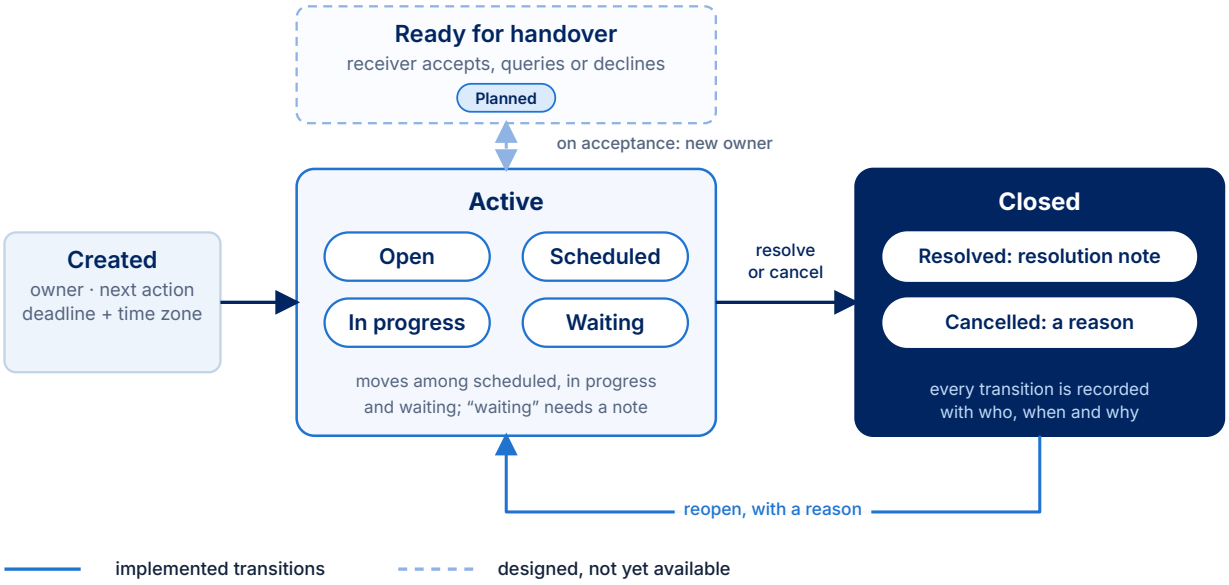


Figure 4. The lifecycle of a continuity record. Solid arrows are implemented transitions; the dashed branch is the designed but not yet available handover.

5.5 Governance of evidence

Relay copies a small piece of customer conversation into another place, so its design treats that copy carefully. These rules are designed to be enforced by the platform itself and are exercised by automated tests.

- **Minimal capture.** Only the messages the user selects are stored, within a bounded size.
- **Access follows the conversation.** An excerpt is shown only to someone who has access to the record and who still has access to the original conversation.
- **Retention and redaction.** Excerpts are hidden a defined period after a record is closed and are then redacted irreversibly.
- **Sensitive content.** Captured messages that look sensitive need an explicit acknowledgement before they are saved.
- **History without content.** Audit events are designed to hold identifiers and changes, not excerpt text or free-text values, so content can be removed without breaking the history.
- **Trash and restore.** A removed source goes to a trash and can be restored. This capability is in development.

5.6 Stage and limits

Table 9. Relay by delivery phase.

PHASE	CONTENT	STAGE
Core persistence and access rules	Records, sources, snapshots, events, ownership rules, retention.	EXISTING
Native selection and conversion	The inbox dialog, type preview, the Relay page and assignment permissions.	EXISTING
Trash and restore	Removal with restore.	IN DEVELOPMENT
Handover	Handover records with acceptance, clarification and decline; end-of-shift bundles.	PLANNED
Scheduling and monitor	Reminders, escalation and the continuity monitor.	PLANNED
External API and browser capture	A token-based API for other clients, and a browser extension that records a selected passage with its page address.	PLANNED
Assisted extraction; company workspaces	Suggested fields from messages; Relay for companies' own support teams.	FUTURE

LIMITATIONS

Relay is currently scoped to the platform's own support inbox and to platform staff; company workspaces are not yet supported. It is not yet linked to tickets, to Workspace panels or to SIE. It sends no reminders, and it produces no automatic summaries. The measures intended to show its value (records created per source, share of active records with an owner and a next action, overdue count and age, handover acceptance time, reminders delivered against failed) are defined but not yet reported, so this paper makes no claim about how much it reduces missed requests.

06

CHAPTER 6

SIE: Support Intelligence Engine

SIE is Mad3oom's specialized engine for technical-support conversations. It reads what a customer writes, weighs it against a catalog of known support situations, and chooses one next action from a closed set while recording why. It is deliberately not a general-purpose chatbot.

6.1 What SIE is, and what it is not

SIE is a deterministic, rule- and data-driven engine. For the same input and the same configuration it produces the same decision: time is supplied as an injectable parameter, there is no randomness, and ties are settled by an explicit rule. Its decision is one action from a closed vocabulary, chosen by ordered rules, and each rule records whether it matched and why.

Determinism and a closed action set make the engine explainable and testable. The trade-off is coverage: SIE handles the situations its catalog describes and does not improvise outside it.

> **WHAT IS NOT CLAIMED**
>
> This paper does not claim human-level reasoning, autonomous resolution of every issue, guaranteed accuracy or independently verified production performance for SIE. A generative model does not make SIE's decisions today. Whether controlled generative assistance could be added is a long-term direction, discussed in Chapter 11.

6.2 Interpreting a problem versus coordinating the work

SIE answers one question: what is this customer telling us, and what should happen next in this conversation? Its possible outputs are conversational: answer from knowledge, ask for more detail, check an understanding with the customer, open a ticket, or hand the conversation to a person. It does not assign owners, set deadlines or track commitments. That coordination belongs to ticketing and to Relay (Chapter 5).

Today SIE's effects stay within the conversation: its chat reply, a ticket created in the same atomic step when it so decides, and handing the conversation to a person. A human takeover of a conversation stops it from replying. There is no link from SIE to Relay yet; handing follow-up work from the engine to Relay is a future possibility.

6.3 A layered design

SIE is organized in nine layers plus a trust boundary that cuts across them. The figure lists them in runtime order, which differs from their numbering: knowledge is attached before the reply is written.

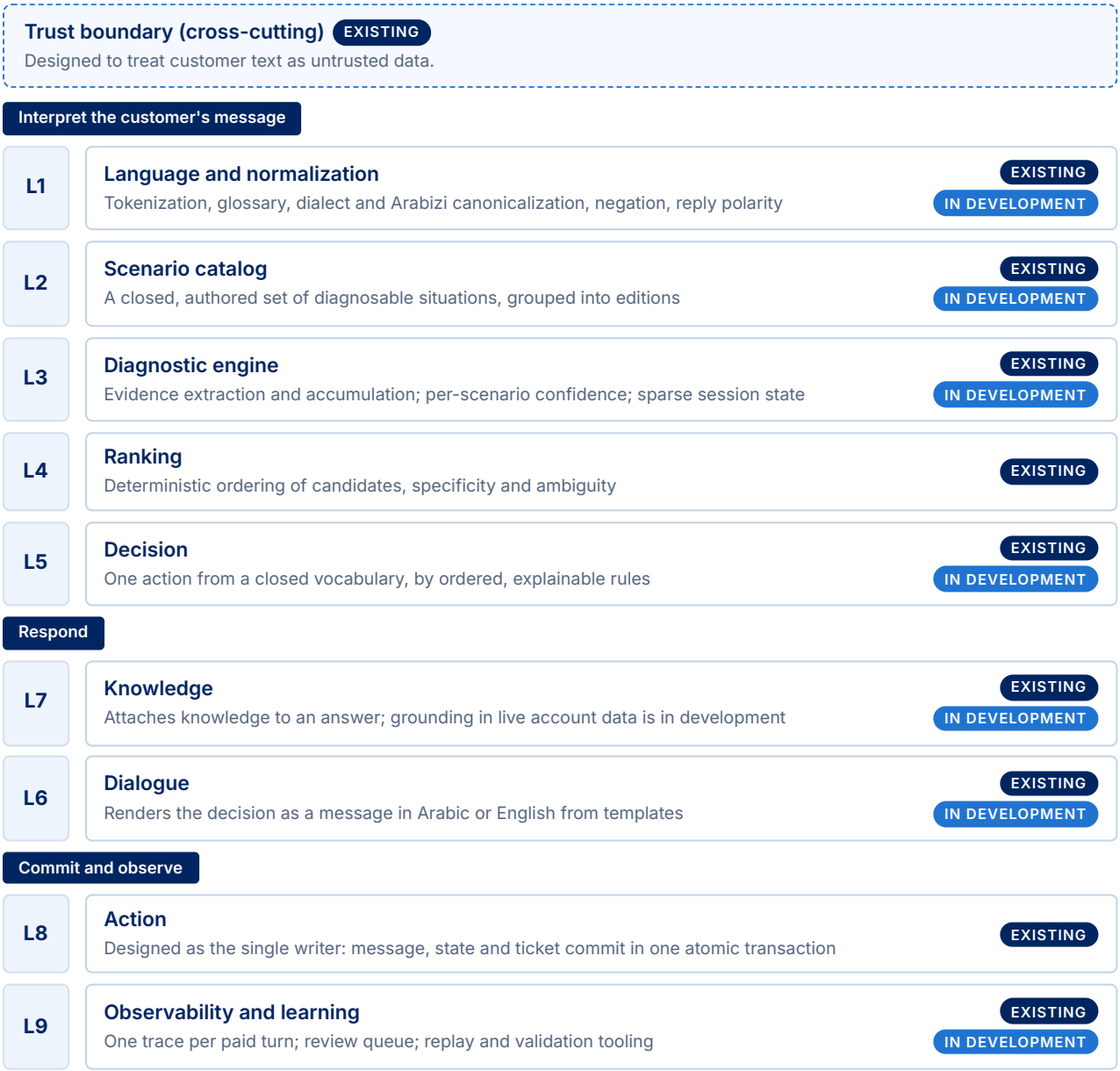


Figure 5. SIE layers in runtime order, with stage. The numbering is the layers' own; the trust boundary is designed to treat customer text as untrusted data across the layers.

Table 10. The seven responsibilities of the engine and what exists for each.

RESPONSIBILITY	WHAT EXISTS
Language understanding and normalization	Tokenization; a technical glossary that maps many phrasings to canonical terms; Arabic-dialect normalization; Arabic written in Latin letters; reply language. Matching respects whole-word boundaries, negation and yes/no polarity.
Scenario-based reasoning and interpretation	A closed catalog of authored scenarios, each with a signature of expected evidence, resolutions and questions; grouped into editions and extensible by a published overlay. About 900 scenarios in the largest edition.

RESPONSIBILITY	WHAT EXISTS
Diagnostic analysis	Extracts evidence from each message, accumulates it over the conversation and scores each candidate scenario by how much of its signature is present. Session state can be stored in a compact sparse form.
Candidate ranking and decision logic	Deterministic ordering with explicit tie-breaks, specificity and an ambiguity flag; then ordered decision rules that choose one action and record every rule evaluated.
Dialogue management	Decisions are rendered as messages in Arabic or English from templates, with a parity test between the two languages.
Knowledge retrieval and application	Static knowledge entries attach to answers for a small set of scenarios. A separate candidate-retrieval step narrows scoring to relevant candidates and is tested to give the same results as a full scan.
Action-oriented support workflows	The action layer is designed to be the single writer. One decision becomes one atomic transaction covering the message, the session state and, when decided, the ticket. Each paid turn is traced once, with the decision's intent and the executed outcome kept apart.

In development

- Grounding answers in live account data (ticket and subscription status) and in published knowledge.
- Continued extension of scenario and language coverage.

Planned

- Awareness of attachments.

6.4 Why the layers matter

- **Modularity.** Dependencies run one way: the engine's decision layers know nothing about channel or platform code, and tests check that vendor details stay inside each channel adapter. Scenarios and glossary are data behind provider interfaces, not code.
- **Testability.** Decision layers are pure functions with an injectable clock, and storage sits behind narrow interfaces, so the suite runs in isolation. At the time of writing all 1,264 tests of the engine pass.
- **Maintainability.** Each layer has its own tests and a narrow contract, writes are designed to go through a single layer, and decisions record the rules they evaluated, which helps trace a surprising reply to its cause.
- **Controlled evolution.** New behaviors are typically introduced behind settings flags whose defaults preserve current behavior.

6.5 What has been verified, and what has not

Table 11. The evidence behind SIE, stated with its limits.

EVIDENCE	WHAT IT SHOWS
Automated tests	1,264 tests of the engine pass (run on 9 October 2026). They verify specified behavior under controlled conditions. They are not accuracy measurements.
Scale benchmarks, synthetic	On generated catalogs, computation time stayed in milliseconds up to 10,000 scenarios; resident memory and index build time, not computation, limit larger sizes. These are synthetic measurements, not production load results.
Trust boundary	Exercised in tests on synthetic hostile and legitimate messages. Its behavior on real traffic has not been measured.
Accuracy, resolution rate, satisfaction	Not measured on real conversations. No such figure is claimed.

6.6 Where SIE runs

SIE runs server-side behind the website chat and the Telegram channel. Telegram reaches it through the shared channel adapter layer (Chapter 7), which is intended to serve every channel. It is offered in editions that differ in scenario coverage, and administrators set its behavior through a settings schema. It is pre-launch.

07

CHAPTER 7

Integrations and Connectivity

Mad3oom's integration vision is that support should meet customers where they already are and connect to the systems teams already use, without letting any single vendor's details leak into the core platform.

7.1 Integration principles

- **Adapters isolate vendors.** Each messaging channel is an adapter with four duties: verify the request, parse the vendor format into one standard message, send replies, and optionally show typing. The engine knows nothing about channels, and a test that reads the source fails if vendor details leak out of the adapter folders.
- **Identity before spend.** A messaging account is linked to a known platform account before the engine replies, so that usage is attributed to the right customer.
- **Verify and de-duplicate before cost.** Verifying incoming requests is a duty of each adapter, and the shared channel pipeline is designed to recognize duplicate deliveries; both happen before anything is charged.
- **Open, standard interfaces.** OAuth 2.1, the Model Context Protocol and token-based APIs, rather than private conventions.

7.2 Channels and connected systems

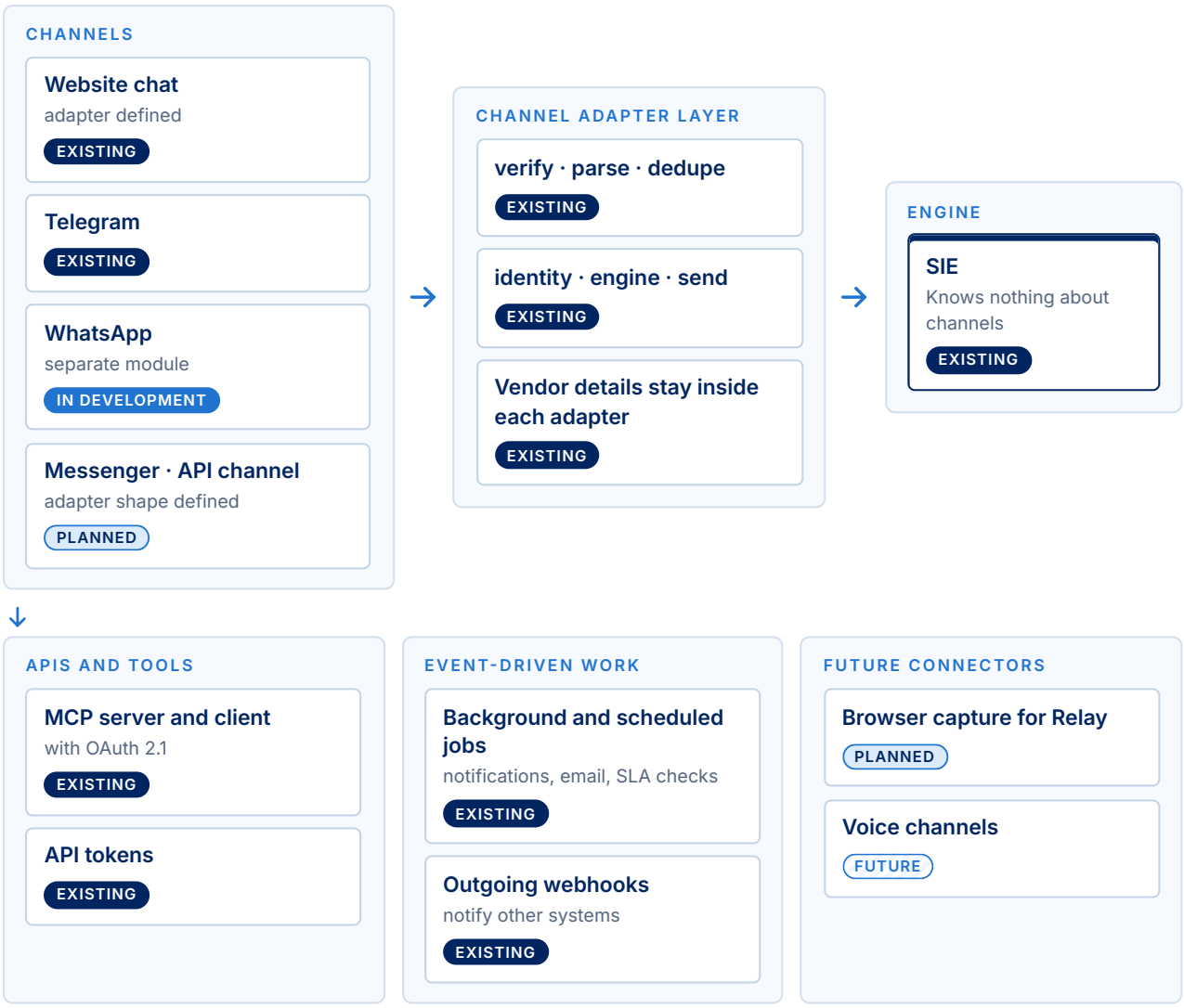


Figure 6. Connectivity map. Channels are intended to reach the engine through one adapter layer; Telegram already does. The lower row shows APIs and tools, event-driven work and future connectors. It is conceptual.

Table 12. Integrations by stage.

INTEGRATION	WHAT EXISTS OR IS INTENDED	STAGE
Website chat and customer portal	A chat widget that runs in the customer's browser under their own session and calls the engine; customer and company dashboards.	EXISTING
Telegram	A support channel handled by the engine, with account linking.	EXISTING
Email	Ticket emails sent through an email provider, and receipt of replies by email.	EXISTING
WhatsApp	A separate WhatsApp module. See the next section; it is not part of the initial launch scope.	IN DEVELOPMENT
Messenger and an API channel	Adapter shapes are defined in the engine's channel layer; no transport is connected.	PLANNED

INTEGRATION	WHAT EXISTS OR IS INTENDED	STAGE
MCP server and client; OAuth 2.1	Mad3oom can expose tools to MCP-compatible AI clients and connect to external MCP servers. Its authorization uses scoped tokens and a page where users revoke connected apps.	EXISTING
API tokens	Authorized account administrators can issue scoped API tokens.	EXISTING
Event-driven work	Background and scheduled jobs produce notifications, email, SLA checks and scheduled replies; an automation builder runs rules when tickets are created; outgoing webhooks notify other systems.	EXISTING
Relay API and browser capture	A token-based API for Relay and a browser extension, using the same authorization server.	PLANNED
Voice channels	Speech-to-text, spoken replies and voice support. This is an exploratory vision and is not on the current schedule.	FUTURE

7.3 WhatsApp Cloud API: three different things

> SCOPE
>
> WhatsApp is not part of Mad3oom's initial launch scope. Nothing in this paper announces it as an available, generally launched customer-facing offering.

Three separate questions are easy to blur, so they are kept apart here.

Table 13. The three layers of the WhatsApp question. The platform's policies change over time and are set by Meta.

QUESTION	WHAT THE EVIDENCE SAYS	STAGE
1. Integration engineering work that exists	A separate WhatsApp module. It connects a business number through Meta's Embedded Signup, stores each inbound message once, runs auto-reply flows that stop when a human takes over, and charges template messages against a prepaid wallet. The module also includes a versioned integration API that lets an approved external application request a pre-approved template message without knowing the provider; its keys can be revoked. Contract tests cover provisioning and the integration API.	IN DEVELOPMENT
2. Technical-provider activities	Meta runs programs for providers that onboard other businesses' accounts, including business verification and app review. Completing them and the commercial terms around them are business and compliance processes that the repositories do not document. This paper therefore states no provider status, partner agreement or approval.	—
3. Future commercial availability	Any general availability would be announced separately and would depend on completing those requirements and on further engineering work.	PLANNED

7.4 What is not claimed

This paper does not claim any partner agreement, customer deployment or integration beyond those in the table above, and it publishes no endpoint addresses or API contracts. Where an integration is listed as planned or in development, it is not part of the generally available offering.

08

CHAPTER 8

Technical Architecture

Mad3oom is built from deliberately simple, modular parts. This chapter describes the architecture at a conceptual level: the layers and the principles that shape them. It does not describe the technology stack, hosting or infrastructure.

8.1 Conceptual architecture

The platform is organized in three layers. People and connected systems reach it through clients and channel adapters. Platform services hold identity and access, the support records and their business rules, integrations and background work. The intelligence layer interprets support conversations and decides the next step.

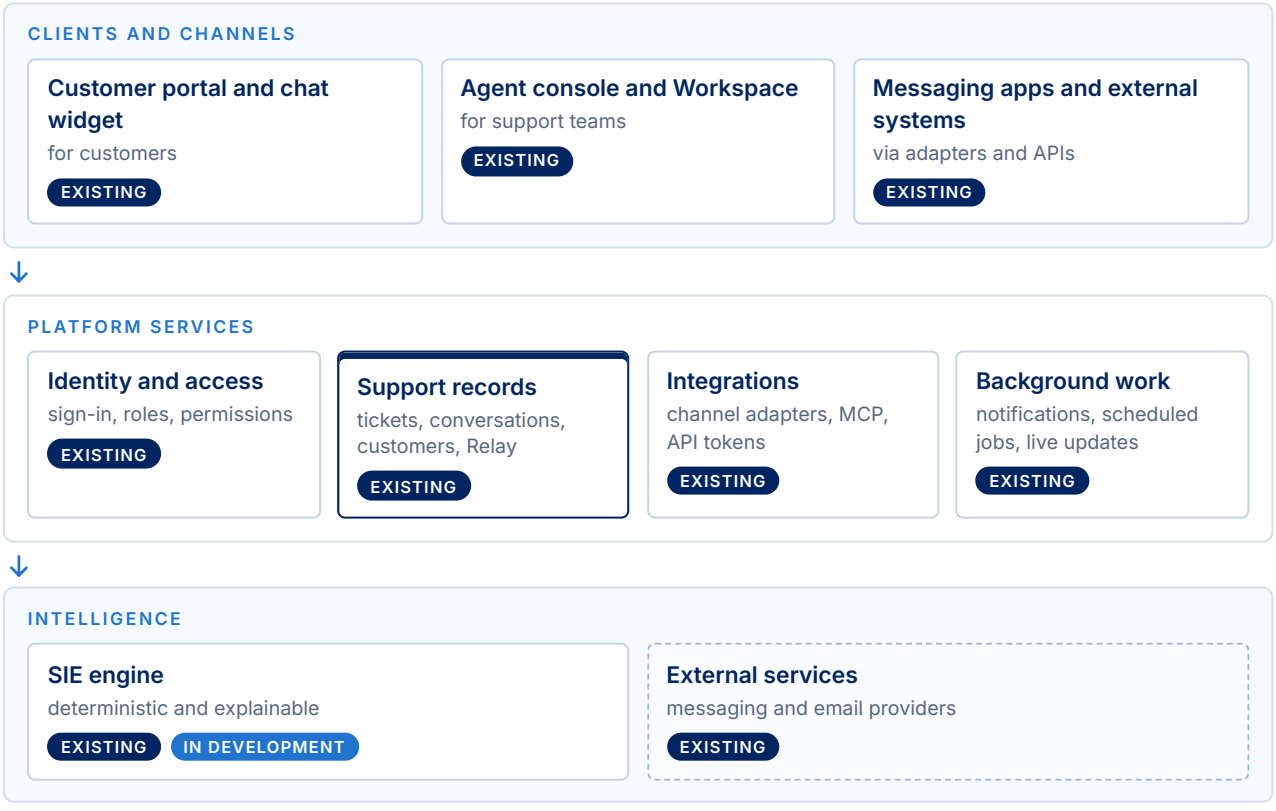


Figure 7. A conceptual overview of the layers. It is not a deployment, network or data-flow diagram.

8.2 Principles at work

Table 14. Architectural principles and where they show in the code.

PRINCIPLE	HOW IT APPEARS TODAY	STAGE
Modularity	Newer features are split into separate model, data and view parts. The engine is layered behind narrow interfaces. Workspace panels are registered, not hard-wired.	EXISTING
Clear interfaces	Channel adapters have a four-duty contract, and the Workspace bridge has a fixed message protocol.	EXISTING
Separation of responsibilities	In SIE, deciding and writing are separate layers, and writes are designed to go through one layer. Workspace does not write support records: the hosted pages do. Domain rules are intended to live with the data rather than in the pages that reach it.	EXISTING
Maintainability and safe change	Changes to data structures are versioned and written to be re-run safely. Relay rejects edits made against a stale version and does not create duplicates when a request is retried.	EXISTING
Reproducible decisions	The engine has no randomness, takes time as an injectable parameter and orders ties explicitly, so decisions are designed to be replayable.	EXISTING
One conversation core	A unified conversation layer intended to be shared by every channel.	IN DEVELOPMENT

8.3 Quality practice

Changes are checked by automated tests and code scanning in continuous integration. The checks include tests of access rules and browser render tests. The SIE engine had 1,264 passing tests on 9 October 2026. Passing tests show specified behavior under controlled conditions; they are not evidence of production reliability.

> **LIMITATIONS**
>
> This public edition does not publish the technology stack, hosting providers, infrastructure, a complete system diagram or API contracts. The architecture is described at the conceptual level only.

09

CHAPTER 9

Security, Privacy and Governance

A customer-support platform holds conversations, identities and operational history. This chapter describes the principles Mad3oom follows to protect that information, the safeguards that can be traced to its code, tests and documented practice, and how security is governed. It states design intent and practice. It does not describe any control as complete or independently verified.

COMPLIANCE AND ASSURANCE STATEMENT

Mad3oom makes no claim of compliance with, or certification against, GDPR, ISO 27001, SOC 2 or any other standard, because none has been independently assessed. No independent penetration test or third-party security audit is documented. The descriptions in this chapter are design objectives and safeguards that can be traced to the product's code, tests and documented practice. They are not guarantees, and they do not mean that every control has been independently verified. For security reasons, the chapter does not publish credentials, customer data or implementation detail.

9.1 Security by design

Mad3oom treats security as part of the architecture rather than as a layer added at the end. Four principles guide design decisions across the platform, Workspace, Relay and SIE.

- **Decide access in the platform, not only in the interface.** Access is decided by the platform itself, so that what a screen shows or hides is never the only control.
- **Grant the least privilege needed.** Use roles, teams and explicit permissions to limit what each person and each component can do.
- **Treat input as untrusted.** Handle customer text and requests that arrive from outside the platform as data to be checked, not as instructions to be followed.
- **Keep only what is needed, for as long as it is needed.** Copy as little customer content as possible, and decide how long it is kept.

9.2 Safeguards and how they are substantiated

For each area, the table gives the design objective and the safeguards that can be traced to the project's code, automated tests and documented practice. “Existing” means present in that code and covered by tests where noted. It does not mean externally audited, and it does not mean that every case is covered.

Table 15. Security areas, design objectives and the safeguards that can be traced to the project.

AREA	DESIGN OBJECTIVE AND SAFEGUARDS	STAGE
Authentication and authorization	Access to account data requires sign-in. Authorization is treated as an architectural responsibility rather than a property of individual pages: access rules are designed to be enforced by the platform, based on the identity of the person making the request.	EXISTING
Least privilege and controlled permissions	Platform staff, company roles and platform ownership are separate authority domains. Support agents work with the conversations assigned to them or to their team, and supervisors have a wider view. In Relay, giving work to another person is an explicit permission, not a default.	EXISTING
Protection of conversations and sensitive information	Access to a conversation determines who can see anything derived from it: Relay shows a saved excerpt only to people who still have access to the original conversation. Relay asks for explicit confirmation before saving captured messages that appear to contain sensitive personal or payment data. SIE is designed to treat customer text as untrusted data.	EXISTING
Secure integrations and credential protection	Integrations use open standards: OAuth 2.1 for application access, scoped tokens, and a page where users revoke connected applications. API tokens can be revoked. Credentials are kept out of the code. Requests that arrive from outside the platform are designed to be verified before they are acted on.	EXISTING
Data minimization, retention and deletion	Relay stores only the excerpts that a user selects, hides them a defined period after a record closes and then redacts them irreversibly; its history keeps identifiers and changes rather than message content. A published process lets people ask for their data to be deleted.	EXISTING
Change management and security review	Changes to data structures are designed to be versioned and safely re-runnable. Automated checks run on changes to the core platform and the engine, including unit and browser tests, tests of access rules, and code scanning. Changes to production are designed to follow a documented approval step.	EXISTING

9.3 Governance and continuous improvement

- Security is considered during development as an internal practice. It is not an independent assessment.
- Security, privacy and operability are treated as part of every capability rather than as a separate phase. Mad3oom has not launched, and a controlled pilot is the intended first step.
- No independent security assessment has taken place, and none is claimed.
- Later editions of this paper will follow the same rule: describe only what can be substantiated.

10

CHAPTER 10

Product Differentiation and Value

Mad3oom's intended differentiation is not a single feature. It comes from combining five design principles in one platform. They are presented here as design intent and intended value; none is presented as a proven result.

10.1 Five design principles

Table 16. Differentiation as design principles, with what exists and what would validate each.

PRINCIPLE	INTENDED VALUE	WHAT EXISTS	WHAT WOULD VALIDATE IT
Structured support management	One structured set of records for tickets, conversations and customers, so context is rebuilt less often.	The core platform. EXISTING	Time to reach full context on a case, measured in real use.
Multi-context work through Workspace	Several contexts open at once, with unsent work protected.	Workspace with local layouts. EXISTING	Observed task time and the rate of lost drafts compared with page-by-page work.
Continuity and follow-up through Relay	Commitments that carry evidence, an owner and a deadline, so that fewer requests are dropped.	Core records and ownership rules. EXISTING Handover and reminders. PLANNED	The instrumented measures listed below.
Specialized support intelligence through SIE	Predictable, explainable and testable behavior that stays inside a closed set of actions.	The core engine. EXISTING Further development under way. IN DEVELOPMENT	Resolution and hand-off quality measured on real conversations, with a defined method.
A modular foundation for integrations	New channels and tools are designed to be added without changing the engine.	The adapter layer, used by Telegram; a website adapter is defined. EXISTING	The effort needed to bring a second messaging channel into production.

10.2 How the parts combine: an illustrative journey

The value of combining the parts is easiest to see along one illustrative case. Steps marked as planned or future do not yet work this way.

Table 17. An illustrative journey. It shows intent, not a recorded customer case.

STEP	WHAT HAPPENS	PART	STAGE
1	A customer describes a problem in the website chat or on Telegram.	Channels	EXISTING
2	SIE interprets the message, answers from knowledge or asks for detail, and opens a ticket if that is its decision. A human takeover stops it.	SIE	EXISTING
3	An agent opens the conversation, the ticket and the customer's history side by side.	Workspace	EXISTING
4	The agent selects the key messages and records a follow-up with an owner, a next action and a deadline.	Relay	EXISTING
5	If the deadline passes, the record shows as overdue in the list.	Relay	EXISTING
6	A reminder is sent before the deadline, and the work is escalated if nobody acts.	Relay	PLANNED
7	At the end of a shift the agent hands the record over, and the colleague must accept it with its context.	Relay	PLANNED
8	The engine hands a follow-up it cannot settle directly to Relay.	SIE, Relay	FUTURE

10.3 Measures that would show the value

Relay's engineering plan defines the measures below to be instrumented from the first day of use. They are defined and not yet reported, so this paper states no figures for them.

- Records created, by type of source.
- The share of active records that have both an owner and a next action.
- The number and age of overdue records.
- The time a handover waits before it is accepted.
- Reminders delivered compared with reminders that failed.

10.4 What is not claimed

> **LIMITATIONS**
>
> Mad3oom is not compared with named competitors here, and no claim of superiority is made. Differentiation is a statement about how the platform is designed. Whether that design produces better outcomes for a given support team is a question for measurement in real use.

11

CHAPTER 11

Product Roadmap

The roadmap is organized by how real each item is today, not by date. It separates what exists, what is being built, what is planned and what remains a long-term opportunity.

READ THIS FIRST

Roadmap items are subject to technical validation and to business priorities. They are not commitments. No delivery dates, customer milestones or completion claims are given, and the order of items within a category is not a schedule.



Figure 8. The four roadmap categories at a glance.

11.1 Existing foundations

EXISTING These are implemented in the repositories. They are foundations to build on, not a claim of production maturity.

AREA	FOUNDATION
Support operations	Tickets with SLA targets and notifications; a helpdesk inbox with teams, assignment, tags, notes, scheduled replies and human–bot handoff; customer history; knowledge base and help center; company accounts, roles and an automation builder.
Workspace	Tabs, groups and splits over six panel types, quick open, unsent-work protection, local layout persistence, keyboard and right-to-left support.
Relay	Continuity records, source evidence with access re-checks, ownership and assignment rules, retention and redaction, an audit log, and the native creation flow from the inbox.
SIE	The nine-layer engine, the trust boundary, candidate retrieval, one trace per paid turn, and the website and Telegram channels.
Integration	The channel adapter layer, MCP with OAuth 2.1, and API tokens.
Engineering	Continuous integration, automated tests of access rules and browser tests.

11.2 Current development and refinement

IN DEVELOPMENT Active work that is not yet complete or available to users.

ITEM	DETAIL
SIE refinement and extension	Continued extension of coverage, and grounding answers in live account data and published knowledge.
Relay trash and restore	In development.
Workspace server-side layouts	In development. Until then layouts are saved per browser.
Conversation Core	A unified conversation layer intended to be shared by every channel; in development.
WhatsApp module	Separate module with its own flows, billing logic and integration API; outside the initial launch scope.

11.3 Planned capabilities

PLANNED Designed and intended, with no delivered implementation yet.

DIRECTION	ITEMS
Relay	Handovers that must be accepted; reminders, escalation and a continuity monitor; an external API and browser capture; Relay as a Workspace panel.
Expanded integrations	WhatsApp, Messenger and an API channel brought onto the shared channel layer; further tool connections through MCP.
SIE	Awareness of attachments.
Workspace	Additional panel types such as knowledge and reports.

11.4 Long-term opportunities

FUTURE Exploratory directions. No commitment is made and none is scheduled.

OPPORTUNITY	WHAT IT WOULD MEAN
Assisted extraction for Relay	Suggested titles, dates and next actions, validated against a strict schema and never confirmed automatically.
Company workspaces in Relay	Relay for companies' own support teams.
Controlled AI-assisted actions	Letting the engine propose or perform approved actions through standard tool interfaces, under policy and with human confirmation where needed.
Richer workspace organization	Named layouts for recurring tasks and context shared across panels.
Shift-aware automation	Automatic handovers once a shift schedule exists as a source in the platform.
Voice channels	Speech-to-text, spoken replies and a voice assistant for agents. An exploratory vision outside the current schedule.

12

CHAPTER 12

Conclusion

Mad3oom's mission is to make technical support more organized, more context-aware, more accountable and more intelligent.

This paper has described how the platform pursues that mission. It brings tickets, conversations, customer records and knowledge into one structured environment. Workspace lets agents work across several contexts without losing what they are doing. Relay keeps follow-up work attached to its evidence, its owner and its deadline. SIE, a layered and explainable engine, interprets what customers write and decides the next step from a closed set of actions.

What exists today is a foundation, not a finished product. The core support features, Workspace, the first parts of Relay and the SIE engine exist in code and are tested. Handovers, reminders, grounded answers, further channels and Relay's external API are still to come, and the platform has not launched. The paper has kept those two statements side by side, because a credible direction depends on being exact about the starting point.

The direction is consistent. Structure comes first, then context, then accountability, then intelligence, each added in a controlled way. The intended path is to validate the design in a controlled pilot before open availability, with security and operability treated as part of every new capability. This paper reports no result that has not been measured, and later editions should hold to the same rule.

No business outcome is guaranteed. What the platform can credibly offer is a disciplined architecture, honest reporting of its own status, and steady progress along the roadmap.

A

APPENDIX A

Capability Status Register

One table that gathers the stage of every capability named in this paper, so that a reader can check a claim in one place. The status reflects the position on 9 October 2026.

Table 18. Capability register. Statuses follow the definitions on page 2.

AREA	CAPABILITY	STAGE	CHAPTER
Core platform	Ticket lifecycle, SLA targets, notifications	EXISTING	3
Core platform	Helpdesk inbox: teams, assignment, tags, notes, scheduled replies, human–bot handoff	EXISTING	3
Core platform	Customer history; knowledge base and help center	EXISTING	3
Core platform	Company accounts, roles, automation builder	EXISTING	3
Workspace	Tabs, groups, splits and quick open over six panel types; unsent-work protection	EXISTING	4
Workspace	Layouts saved per browser	EXISTING	4
Workspace	Layouts saved per agent on the server	IN DEVELOPMENT	4
Workspace	Relay and further panel types	PLANNED	4
Workspace	Adaptive, named layouts and shared context	FUTURE	4
Relay	Follow-up and issue records, lifecycle, categories, creation from inbox messages	EXISTING	5
Relay	Ownership and assignment rules; derived overdue flag; audit events	EXISTING	5
Relay	Evidence governance: access re-checks, retention, irreversible redaction	EXISTING	5
Relay	Trash with restore	IN DEVELOPMENT	5
Relay	Handovers with acceptance, clarification and decline	PLANNED	5
Relay	Reminders, escalation, continuity monitor	PLANNED	5
Relay	External API; browser capture	PLANNED	5
Relay	Assisted extraction; company workspaces	FUTURE	5
SIE	Nine layers: language, scenarios, diagnostics, ranking, decision, dialogue, knowledge, action, observability	EXISTING	6
SIE	Trust boundary; candidate retrieval; sparse session state	EXISTING	6

AREA	CAPABILITY	STAGE	CHAPTER
SIE	Continued extension of SIE	IN DEVELOPMENT	6, 11
SIE	Grounding in live account data and published knowledge	IN DEVELOPMENT	6
SIE	Handing follow-up work to Relay	FUTURE	6
SIE	Controlled generative or tool-using assistance	FUTURE	6, 11
Integrations	Channel adapter layer; website chat and Telegram	EXISTING	7
Integrations	MCP, OAuth 2.1, API tokens	EXISTING	7
Integrations	Trigger- and schedule-driven side effects	EXISTING	7
Integrations	WhatsApp module and its integration API (separate; outside the initial launch scope)	IN DEVELOPMENT	7
Integrations	WhatsApp, Messenger and API channel on the shared layer	PLANNED	7
Integrations	Voice channels	FUTURE	7
Architecture	Platform-enforced access rules, continuous integration	EXISTING	8, 9
Architecture	Conversation Core: a unified conversation layer for all channels	IN DEVELOPMENT	8
Security	Security design objectives and traceable safeguards, as described in Chapter 9; not independently assessed	EXISTING	9

B

APPENDIX B

Glossary

TERM	MEANING IN THIS PAPER
Action layer	The SIE layer designed to be the single writer: it commits a decision's message, state and ticket in one atomic transaction.
Adapter (channel)	A module that hides one messaging vendor's details behind a common contract: verify, parse, send.
Arabizi	Arabic written in Latin letters and digits, as often used in chat.
Continuity record	Relay's unit of work: evidence, ownership, a next action, a deadline and an append-only history.
Conversation Core	A unified conversation layer intended to be shared by all channels; in development.
Deterministic	Producing the same output for the same input and configuration; no randomness, injectable time, explicit tie-breaking.
Evaluated rules	The record, kept with each decision of SIE's decision layer, of every rule that was checked, whether it matched and why.
Idempotency key	A client-chosen identifier that makes a retried request return the first result instead of creating a duplicate.
Least privilege	Giving each person and component only the access needed for their task.
MCP	Model Context Protocol, a standard way for AI clients to use tools and data exposed by a server.
OAuth 2.1	An open standard for delegated, scoped authorization of applications.
Optimistic concurrency	Rejecting an edit made against a stale version of a record instead of overwriting newer changes.
Panel	One tab in Workspace, hosting a page or a single record.
Scenario	An authored description of a support situation in SIE's catalog: expected evidence, resolutions and questions.
SIE	Support Intelligence Engine, Mad3oom's deterministic engine for interpreting support conversations.
SLA	Service-level agreement; here, a target time for the first response to a ticket, by priority.
Trust boundary	The component of SIE designed to treat customer text as untrusted data wherever it could influence the engine.

C

APPENDIX C

Basis of Preparation and Limitations

How the statements in this paper were established, and what was not verified. A white paper is only as credible as its evidence.

C.1 Sources reviewed

- The project's source code, engineering documentation and automated tests, as they stood on 9 October 2026.

C.2 Method

- Statements about what exists rest on the code and on the engineering records.
- On 9 October 2026, while this paper was prepared, the SIE engine's automated suite was run on the repository's latest commit, and 1,264 tests passed.
- Figures from engineering records, such as test counts, code sizes and timings, are quoted with their date and with what they measure.
- The English and Arabic editions are produced from one source, so that their structure and facts match. The Arabic edition is equivalent in meaning, not a word-for-word translation.

C.3 What was not verified

NOT VERIFIED	CONSEQUENCE FOR THIS PAPER
Production usage and volumes	No customer counts, usage figures or deployments are reported.
Availability of each capability to end users	“Existing” means implemented in the repositories. It is not a statement that the capability is available to end users.
Real-world accuracy of SIE	No accuracy, resolution or satisfaction figures are reported.
Security assessment by a third party	No certification or compliance is claimed.
Independent verification of security controls	Not performed. Security descriptions are design objectives and safeguards traceable to the project, not independently verified guarantees.
Meta provider status and commercial terms	No partnership or provider status is stated.
Changes after 9 October 2026	They are not reflected in this edition.

C.4 Editorial rules

- Every capability carries a stage label, and a planned feature is never described as a current capability.
- Passing automated tests are never presented as proof of production reliability.
- No competitor is named and no superiority is claimed. No credentials, customer data or exploitable security detail is included.
- This public edition describes the platform at a conceptual level and does not publish operational or implementation detail.

C.5 Document history

VERSION	DATE	NOTE
1.0	October 2026	Foundational edition. Information as of 9 October 2026.